Digital VLSI Design

4-bit Array Multiplier from RTL to GDSII

with Open-Source Tools on SkyWater SKY130

Team **Cold Cold Drinks**

Name	**Roll number**
Allu Joanna Preetika	2024112008
Sindhuja Babu	2024102055

Repository: https://github.com/Sindhuja212006/multiplier4

October 2026

Contents

1 Introduction **2**

2 Design and Verification (Part A) **2**
2.1 Architecture . . . 2
2.2 Testbench and simulation results . . . 3
2.3 Hand-worked example . . . 3
2.4 Waveform and latency . . . 4

3 Logic Synthesis (Part B) **4**

4 Static Timing Analysis (Part C) **5**

5 Physical Design (Part D) **6**
5.1 Runs . . . 7
5.2 Results . . . 7
5.3 Layout . . . 8

6 Answers to Questions **10**
6.1 Logic cells versus 16 AND gates and 12 full adders . . . 10
6.2 Critical path . . . 10
6.3 Flip-flop area . . . 11
6.4 Post-route $F_{\max}$ and added cells . . . 11
6.5 DRC and LVS . . . 11
6.6 Scaling to 8 bits . . . 11

7 Conclusion **12**

8 Contributions **12**

1 Introduction

This report describes the design of a 4-bit × 4-bit unsigned array multiplier and its implementation from RTL to a final GDSII layout on the SkyWater SKY130 130 nm process, using the `sky130_fd_sc_hd` standard cell library and only open-source tools.

The multiplier registers both inputs and its 8-bit output, so it has a latency of two clock cycles and a throughput of one result per cycle. The exact commands for every step are listed in the repository `README.md`.

2 Design and Verification (Part A)

2.1 Architecture

The multiplier follows the shift-and-add array structure. Each bit b_i of B produces a partial product $pp_i[j] = a[j] \cdot b[i]$, giving $4 \times 4 = 16$ AND gates. Row 0 needs no adders: its lowest bit is the product bit $p[0]$ and its other three bits are passed down. Rows 1, 2 and 3 are each a 4-bit ripple-carry adder built from 4 full adders, for 12 full adders in total. The lowest sum bit of each row is a finished product bit ($p[1]$, $p[2]$, $p[3]$), and row 3 also produces $p[7{:}4]$.

Inputs `a` and `b` are captured into 4-bit registers (`a_r`, `b_r`) and the 8-bit result into register `p`, all with an asynchronous active-low reset `rst_n`. This gives exactly 16 flip-flops. Figure 1 shows the complete structure.

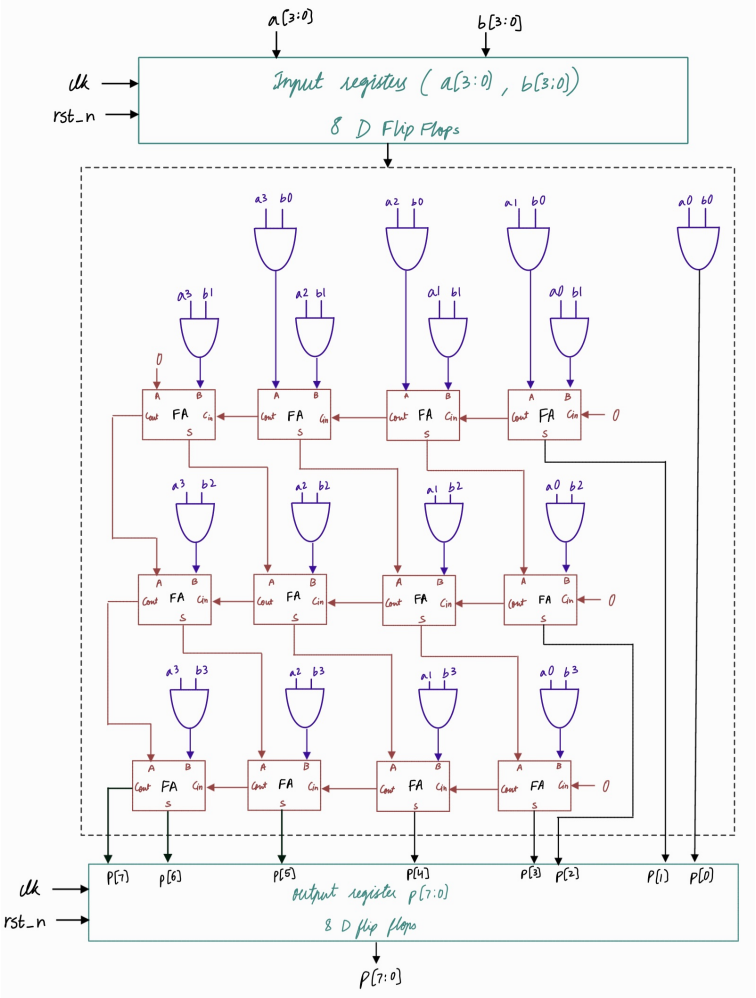


Figure 1: Block diagram of `mult_array`: input registers (8 D flip-flops), 16 AND gates, 12 full adders in three ripple-carry rows, and the output register (8 D flip-flops).

The full adder is written with continuous assignments (Listing 1). `mult_array.v` instantiates it 12 times using `generate` loops, wired exactly as in the connection table of the assignment guide; the `*` operator is not used.

Listing 1: `rtl/full_adder.v`

```
module full_adder (
    input wire a, b, cin,
    output wire sum, cout
);
    assign sum = a ^ b ^ cin;
    assign cout = (a & b) | (cin & (a ^ b));
endmodule
```

2.2 Testbench and simulation results

The testbench `tb/tb_mult.v` generates a 10 ns clock, holds `rst_n` low for the first two cycles, and then applies all 256 input combinations. For each one it waits for the two-cycle latency, compares `p` with `a * b`, prints a line for every mismatch and finally prints one summary line. It also writes the waveform file `mult.vcd`.

The RTL passes all 256 vectors (Figure 2).

```
vvp sim
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:52: $finish called at 5140000 (1ps)
```

Figure 2: RTL simulation output: `PASS: 256 vectors checked, 0 errors`.

To confirm that the testbench really catches errors, the sum in `full_adder.v` was temporarily changed to `a ^ b` (dropping `cin`). The testbench then reported 95 mismatches and printed `FAIL` (Figure 3). The change was reverted afterwards and the RTL passed again.

```
MISMATCH at 5080000 ns: a=15 b=12 p=196 expected=
MISMATCH at 5100000 ns: a=15 b=13 p=139 expected=
MISMATCH at 5120000 ns: a=15 b=14 p=154 expected=
MISMATCH at 5140000 ns: a=15 b=15 p=181 expected=
FAIL: 256 vectors checked, 95 errors
tb/tb_mult.v:52: $finish called at 5140000 (1ps)
```

Figure 3: Error test with `sum = a ^ b`: the last mismatch lines and `FAIL: 256 vectors checked, 95 errors`. (The `%t` format prints times in the 1 ps simulator precision, so "5140000" means 5140 ns.)

2.3 Hand-worked example

The example uses Allu Joanna Preetika's roll number, 2024112008. Its last digit is 8, so $A = 8 + 5 = 13 = 1101_2$ and $B = 7 = 0111_2$. The expected product is $13 \times 7 = 91 = 01011011_2$. The partial products are $pp_0 = pp_1 = pp_2 = 1101$ (since $b_0 = b_1 = b_2 = 1$) and $pp_3 = 0000$ (since $b_3 = 0$), shifted by 0, 1, 2 and 3 places (Figure 4): $13 + 26 + 52 + 0 = 91$.

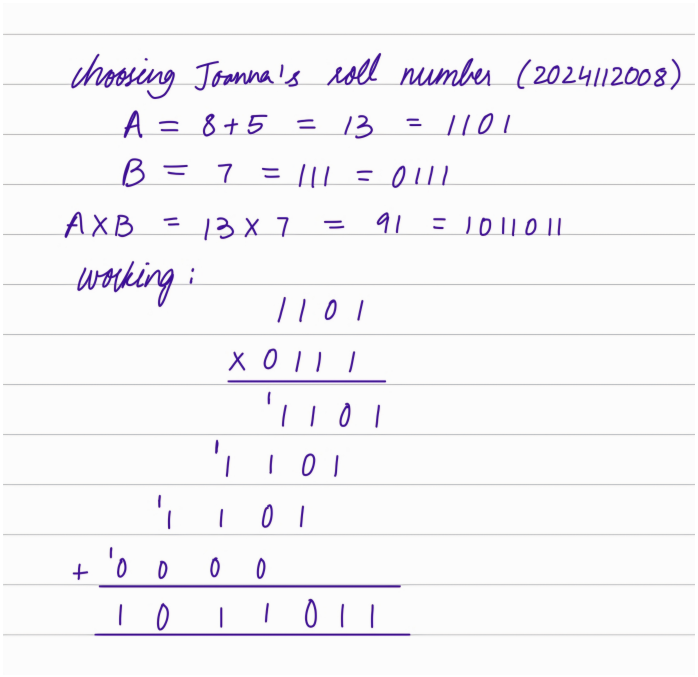


Figure 4: Hand-worked multiplication of $A = 13$ and $B = 7$.

2.4 Waveform and latency

Figure 5 shows the same multiplication in the simulation waveform (values in hexadecimal: `D` = 13, `4E` = 78, `5B` = 91, `68` = 104).

- **Marker A, 4320 ns**: the testbench applies $a = 13$, $b = 7$ on a falling clock edge.
- **Marker B, 4325 ns**: rising edge 1; a and b are captured into the input registers. `p` still shows the previous result, $13 \times 6 = 78$.
- **Marker C, 4335 ns**: rising edge 2; the product is captured into the output register and `p` becomes `5B` = 91.

The product therefore appears two clock cycles after the inputs are applied, which confirms the two-cycle latency. The same pattern repeats for the next vector ($b = 8$ at 4340 ns, $p = 104$ at 4355 ns).

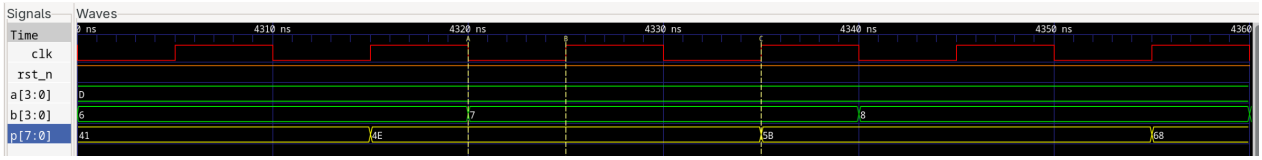


Figure 5: RTL simulation of $A = 13$, $B = 7$. Marker A: inputs applied (4320 ns); marker B: clock edge 1, inputs registered (4325 ns); marker C: clock edge 2, $p = 0x5B = 91$ (4335 ns).

3 Logic Synthesis (Part B)

`synth/synth.tcl` reads both RTL files, runs generic synthesis with the full adders flattened into one module, maps the flip-flops and then the logic to SKY130 cells (`dfflibmap`, `abc` with the typical-corner Liberty file `tt_025C_1v80`), removes unused wires, and writes the cell report `synth/mult_array_stat.rpt` and the netlist `synth/mult_array_netlist.v`. Table 1 summarises the results.

Table 1: Part B synthesis results (`synth/mult_array_stat.rpt`).

Metric	Value
Total number of cells	68
Number of flip-flops	16 (`sky130_fd_sc_hd__dfrtp_1`)
Total cell area	$785.754\,\mu m^2$
Flip-flop area	$16 \times 25.024 = 400.384\,\mu m^2$
Flip-flop area (% of total)	50.96 %
3 most-used cell types	`dfrtp_1` ×16, `nand2_1` ×8, `and2_0` and `xnor2_1` ×6 each
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

The 52 logic cells (68 total minus 16 flip-flops) are listed in Table 2. The synthesised netlist was simulated with the same testbench and the SKY130 cell models, and it passed all 256 vectors, so synthesis did not change the behaviour.

Table 2: Logic cells in the synthesised netlist (excluding the 16 flip-flops).

Cell	Count	Cell	Count
`nand2_1`	8	`a2111oi_0`	1
`and2_0`	6	`a21oi_1`	3
`xnor2_1`	6	`a22oi_1`	3
`maj3_1`	5	`o21ai_0`	2
`nor2_1`	4	`mux2i_1`	2
`and4_1`	2	`clkinv_1`	2
`xnor3_1`	2	`lpflow_isobufsrc_1`	2
`xor3_1`	1	`nand3_1`	1
`or3_1`	1	`nand4_1`	1
Total logic cells			**52**

4 Static Timing Analysis (Part C)

`sta/sta.tcl` reads the typical-corner Liberty file and the netlist, defines the clock on `clk` with period T, sets input and output delays of 1.0 ns, reports the worst setup slack and saves the critical path to `sta/mult_array_timing.rpt`.

With $T = 10$ ns the worst slack was WNS $= 7.027$ ns, so

$$T_{\min} = T - \mathrm{WNS} = 10 - 7.027 = 2.973\,\mathrm{ns}.$$

$T_{\min}$ was rounded *up* to 2.98 ns (rounding to 2.97 ns would leave a slightly negative slack). The re-run at $T = 2.98$ ns gave a slack of 0.007 ns, close to zero as expected, and

$$F_{\max} = \frac{1000}{T_{\min}} = \frac{1000}{2.98} = 335.6\,\mathrm{MHz}.$$

Table 3: Part C timing results (typical corner `tt_025C_1v80`).

Metric	Value
Worst slack at $T = 10\,\text{ns}$	7.027 ns
$T_{\min}$	2.98 ns
$F_{\max}$	335.6 MHz
Slack at $T = T_{\min}$	0.007 ns
Critical path start point	`_108_` (`dfrtp_1`, input register `b_r[0]`)
Critical path end point	`_103_` (`dfrtp_1`, output register `p[7]`)
Number of cells on the critical path	9 (including the launching flip-flop)
Latency $= 2 \times T_{\min}$	5.96 ns
Throughput	335.6 million results per second

The critical path runs from the input register `b_r[0]` to the output register bit `p[7]`, which captures the final carry `c3[4]`. Table 4 lists the cells on it, in order.

Table 4: Cells on the critical path at $T = 2.98\,\text{ns}$ (`sta/mult_array_timing.rpt`).

#	Instance	Cell	Delay (ns)	Arrival (ns)
1	`_108_`	`dfrtp_1` (CLK→Q)	0.410	0.410
2	`_050_`	`and4_1`	0.223	0.634
3	`_061_`	`o21ai_0`	0.195	0.828
4	`_062_`	`nand3_1`	0.140	0.969
5	`_063_`	`maj3_1`	0.380	1.349
6	`_069_`	`xnor3_1`	0.393	1.741
7	`_074_`	`maj3_1`	0.369	2.110
8	`_086_`	`maj3_1`	0.383	2.493
9	`_087_`	`maj3_1`	0.349	2.842
Setup time of `_103_`: 0.131 ns; required time 2.849 ns				

Five of the cells come from full adders: four `maj3` cells (full-adder carry) and one `xnor3` cell (full-adder sum). The `and4`, `o21ai` and `nand3` cells are partial-product AND logic that synthesis merged with the first adder stage. This is compared with the $2N = 8$ full-adder estimate in Section 6.2.

5 Physical Design (Part D)

LibreLane was run on the RTL with the configuration in Listing 2. The starting clock period was $1.2 \times T_{\min} = 1.2 \times 2.98 = 3.58\,\text{ns}$, rounded up to the next 0.5 ns, i.e. 4.0 ns. Because the container's default PDK is not SKY130, the PDK and library were passed on the command line (`librelane -manual-pdk -pdk-root /foss/pdks -pdk sky130A -scl sky130_fd_sc_hd config.json`).

Listing 2: `pnr/config.json` (final run)

```
{
    "DESIGN_NAME": "mult_array",
    "VERILOG_FILES": ["dir::../rtl/full_adder.v", "dir::../rtl/mult_array.v"],
    "CLOCK_PORT": "clk",
    "CLOCK_PERIOD": 6.0,
    "FP_SIZING": "absolute",
    "DIE_AREA": [0, 0, 150, 150]
}
```

5.1 Runs

Every run was DRC- and LVS-clean, but the first four failed setup timing in the slow corner (`ss_100C_1v60`). After each failure the clock period was increased by 0.5 ns (Table 5). The fifth run, at 6.0 ns, passed all checks.

Table 5: LibreLane attempts. Setup slack is the worst value over all corners (`timing__setup__ws`).

#	Run folder	Period (ns)	Setup slack (ns)	DRC	LVS	Outcome
1	`RUN_2026-10-23_18-59-28`	4.0	−1.688	0	0	setup violation
2	`RUN_2026-10-23_19-04-48`	4.5	−1.188	0	0	setup violation
3	`RUN_2026-10-23_19-08-06`	5.0	−0.688	0	0	setup violation
4	`RUN_2026-10-23_19-15-56`	5.5	−0.188	0	0	setup violation
5	`RUN_2026-10-23_19-19-43`	6.0	0.044	0	0	**passed**

5.2 Results

Table 6: Part D results (`pnr/metrics.json`, run 5).

Metric	Value
`CLOCK_PERIOD` used	6.0 ns
Number of runs needed	5
Core area	17 759.5 µm^2 (die 22 500 µm^2)
Cells after synthesis (Part B)	68
Cells after place and route	1891
Kinds of cells added	decap, fill, tap, timing-repair buffers, clock buffers
Wirelength (routed)	2407 µm
Setup slack (worst corner)	0.044 ns
Post-route $F_{\max}$	$1000/(6.0 - 0.044) = 167.9$ MHz
$F_{\max}$ from Part C	335.6 MHz
DRC errors	0 (Magic, KLayout and router)
LVS errors	0

Other values from the final run: worst hold slack 0.139 ns, setup slack at the typical corner 2.914 ns, and standard-cell utilisation 7.3 %.

The layout contains $1891 - 68 = 1823$ more cells than the synthesised netlist. Table 7 breaks them down; the decap and fill split was counted from the final DEF file, since `metrics.json` reports them together as `fill_cell` = 1552. LibreLane runs its own synthesis, so the logic cells in the layout are not identical to the Part B netlist (for example it uses 12 `nand2_1` cells instead of 8), although the logic count is again 68.

Table 7: Cells added by the physical design flow.

Kind	Cells	Count	Purpose
Decap	`decap_3/4/6/8/12`	1300	Capacitance between power and ground to smooth supply noise
Fill	`fill_1`, `fill_2`	252	Fill gaps in the rows so wells and power rails stay continuous
Tap	`tapvpwrvgnd_1`	245	Tie the N-well and substrate to power and ground, preventing latch-up
Timing repair	`buf_2`, `buf_4`, `clkdlybuf4s50_1`	23	Drive weak or long nets to meet setup and slew limits
Clock tree	`clkbuf_16`	3	Balance clock arrival at the 16 flip-flops
Total added		**1823**	

5.3 Layout

Figure 6 shows the whole 150 µm × 150 µm die and Figure 7 a zoomed view of the standard cell rows and metal wires.

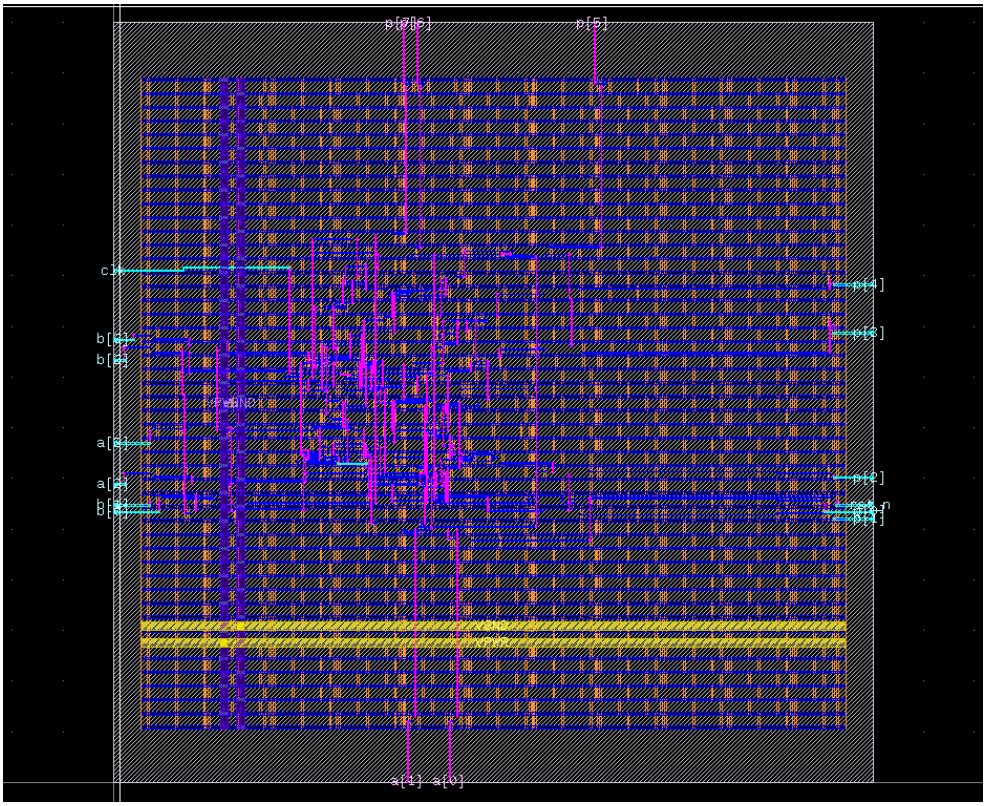


Figure 6: Final layout of `mult_array`, whole die (150 µm × 150 µm, top-level view): I/O pins on the edges, the power grid straps (VPWR/VGND), the standard cell row rails, and the signal routing clustered in the centre of the core.

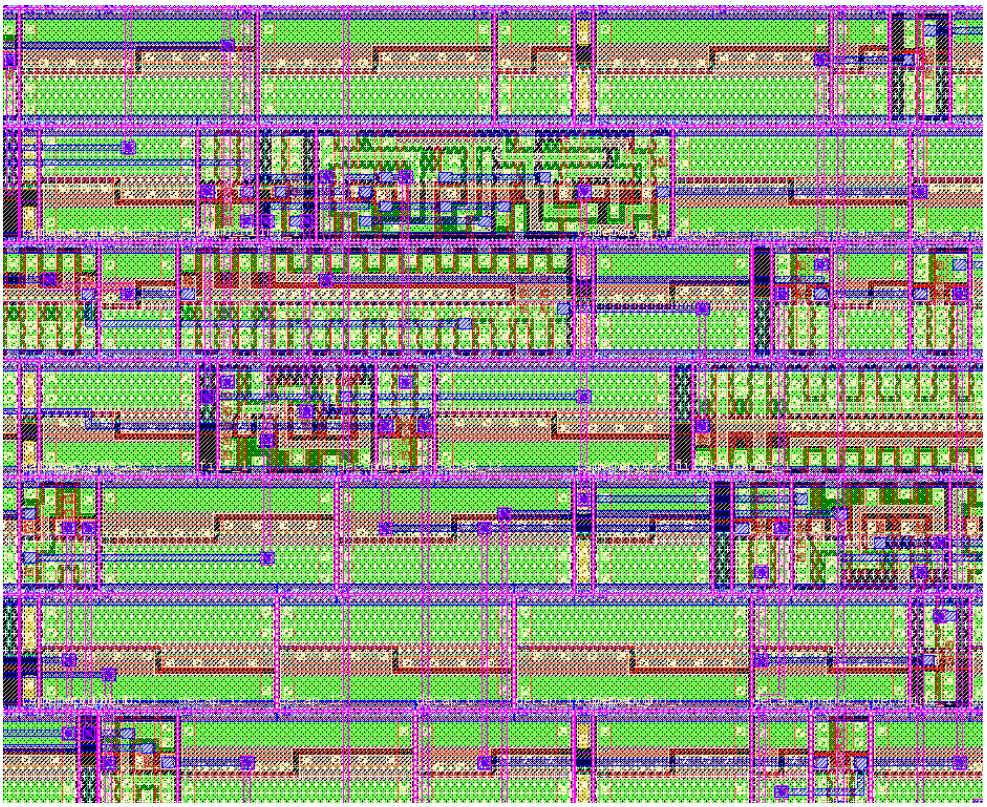


Figure 7: Zoomed view of the layout (full hierarchy): standard cell rows separated by power rails, logic cells next to decap and fill cells, horizontal met1 and vertical met2 wires, and the vias between them.

For clarity, we also took two additional views of the same layout: the whole die with every cell's contents drawn (Figure 8), and the zoomed region in the top-level view, where the cell names are visible (Figure 9).

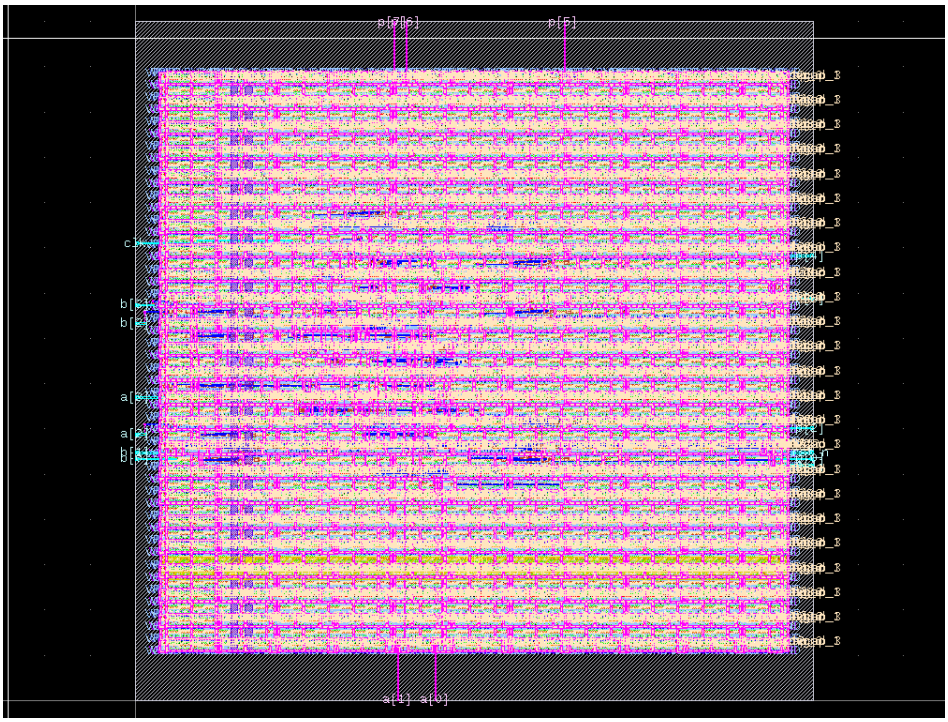


Figure 8: Whole die with all cell contents drawn (full hierarchy).

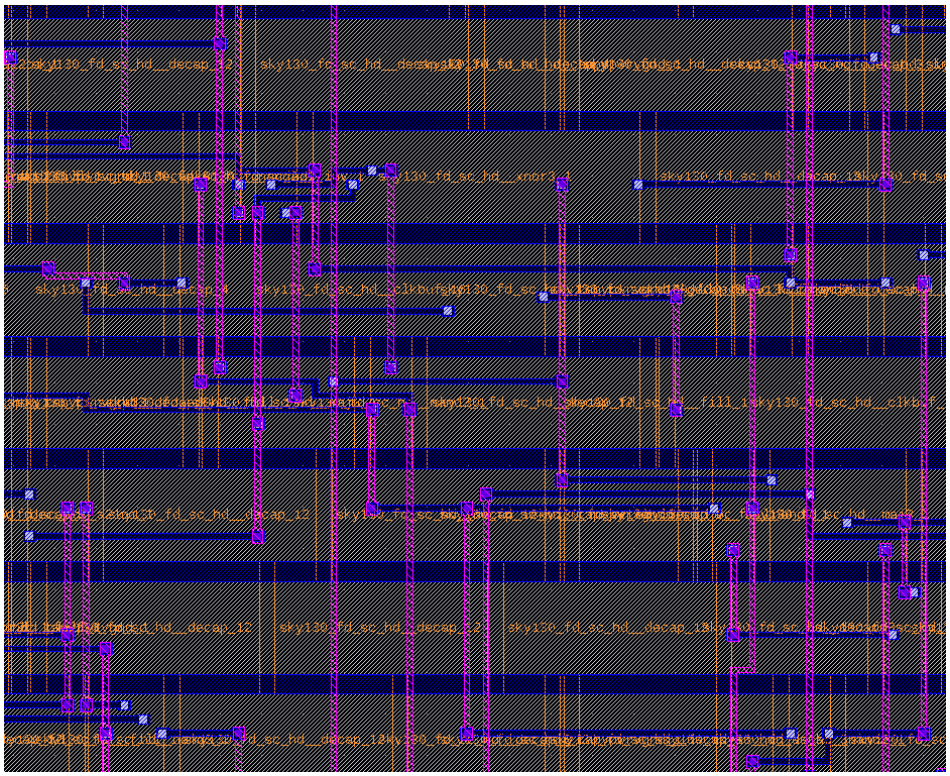


Figure 9: Zoomed routing (top-level view) with cell names visible, e.g. `decap_12`, `fill_1`, `xnor3_1` and `clkbuf_16`.

6 Answers to Questions

6.1 Logic cells versus 16 AND gates and 12 full adders

Synthesis produced 68 cells, of which 16 are flip-flops, so the logic uses $68 - 16 = 52$ cells, not $16 + 12 = 28$. A full adder is not a single SKY130 cell: its sum needs a 3-input XOR and its carry a majority function, so most full adders map to two or more cells (`xnor2`, `xnor3`, `xor3`, `maj3`, `nand2`, AOI/OAI gates). Because the design is flattened, ABC also optimises across the full-adder boundaries: some of the 16 AND gates are absorbed into complex cells such as `a22oi`, `a21oi` and `and4` (only six `and2` cells remain), and the four adders with a constant-zero input ($c_i[0] = 0$ in each row and $r_0[3] = 0$) simplify to half adders. A few extra cells (two `clkinv` and two `lpflow_isobufsrc`) provide inversions and buffering. The number of cells therefore reflects the cell library and the optimisation, not the number of logical blocks.

6.2 Critical path

The critical path starts at the input register `b_r[0]` (`_108_`) and ends at the output register bit `p[7]` (`_103_`), which captures the final carry `c3[4]`. It has 9 cells: the launching flip-flop, three partial-product/first-stage cells (`and4`, `o21ai`, `nand3`) and five full-adder cells (four `maj3` and one `xnor3`), with 2.842 ns arrival time. The estimate for a 4-bit array is about $2N = 8$ full-adder delays. The path has only 5 full-adder cells because synthesis implements a full-adder carry in a single `maj3` cell, merges the AND gates into the first adder stage, and simplifies the adders with constant-zero inputs. The shape matches the estimate, though: the path goes down through the rows and then ripples along the carry chain of row 3, and the 8 combinational cells take $2.842 - 0.410 = 2.432$ ns, about 0.3 ns per stage.

6.3 Flip-flop area

The 16 flip-flops occupy 400.384 µm^2 of 785.754 µm^2, i.e. 50.96 % of the cell area. A `dfrtp_1` cell (D flip-flop with asynchronous reset) is 25.024 µm^2, about 6.7 times a `nand2_1` (3.75 µm^2), because it contains two latches, clock buffering and reset logic. The design has 16 of these against only 52 small logic cells, so the registers dominate. The percentage falls for larger multipliers: the logic grows as N^2 (AND gates) and $N(N-1)$ (full adders), but the number of flip-flops grows only as $4N$.

6.4 Post-route $F_{\max}$ and added cells

The post-route $F_{\max}$ is $1000/(6.0 - 0.044) = 167.9$ MHz, about half of the 335.6 MHz from Part C. There are two reasons. First, LibreLane signs off at the worst process corner, `ss_100C_1v60` (slow transistors, 100 °C, 1.60 V), while Part C used only the typical corner. Second, after routing the timing includes the real wire resistance and capacitance, the clock tree and the inserted buffers, while Part C assumed ideal wires and an ideal clock. The corner is the larger effect: at the typical corner the post-route slack is 2.914 ns, which corresponds to a period of $6.0 - 2.914 = 3.086$ ns (324 MHz), close to the Part C value.

The flow added 1823 cells (Table 7): 1300 **decap** cells, which act as local charge reservoirs that reduce supply noise when cells switch; 252 **fill** cells, which keep the wells, implants and power rails continuous in the gaps between cells so the layout passes DRC; 245 **tap** cells, which connect the N-well and substrate to VPWR and VGND at regular spacing to prevent latch-up; 23 **timing-repair buffers**, which drive long or heavily loaded nets so that slew and setup limits are met; and 3 **clock buffers** (`clkbuf_16`), inserted by clock tree synthesis so the clock reaches all 16 flip-flops at nearly the same time. The very high decap and fill count comes from the low utilisation (7.3 %) of the fixed 150 µm × 150 µm die.

6.5 DRC and LVS

DRC (design rule check) verifies that the layout geometry obeys the foundry's manufacturing rules: minimum widths and spacings of each layer, enclosures of vias and contacts, overlaps, and density rules. LVS (layout versus schematic) extracts a netlist of devices and connections from the layout and checks that it matches the intended netlist, so that every cell is present and every net is connected correctly with no shorts or opens. Both are needed: a DRC-clean layout may still implement the wrong circuit, and a correct circuit that breaks design rules cannot be fabricated reliably and may have low yield. Our final layout has 0 DRC errors (Magic, KLayout and router) and 0 LVS errors (Netgen).

6.6 Scaling to 8 bits

An N-bit array multiplier needs N^2 AND gates and $N(N-1)$ full adders, and its critical path is about $2N$ full-adder delays.

	$N = 4$	$N = 8$
AND gates, N^2	16	64
Full adders, $N(N-1)$	12	56
Flip-flops, $4N$	16	32
Critical path, $\approx 2N$ FA delays	8	16

The number of full adders grows from 12 to 56, i.e. $56/12 \approx 4.7$ times, while the critical path only doubles. Using our Part C result, the combinational part of the critical path takes 2.432 ns for an estimated 8 full-adder delays, i.e. about 0.30 ns per full-adder delay. For 8 bits:

$$T_{\min} \approx t_{\text{clk-q}} + 16 \times 0.30 + t_{\text{setup}} = 0.41 + 4.86 + 0.13 \approx 5.4\,\text{ns},$$

giving $F_{\max} \approx 1000/5.4 \approx 185\,\text{MHz}$ at the typical corner before layout, compared with 335.6 MHz for 4 bits. Area grows roughly quadratically, delay roughly linearly.

7 Conclusion

The 4-bit array multiplier was designed structurally from 16 AND gates and 12 full adders with registered inputs and outputs, and it passed all 256 input combinations in both RTL and gate-level simulation. Synthesis produced 68 cells with $785.754\,\mu\text{m}^2$ of cell area, half of it in the 16 flip-flops. Static timing analysis at the typical corner gave $T_{\min} = 2.98\,\text{ns}$ ($F_{\max} = 335.6\,\text{MHz}$), with the critical path running from `b_r[0]` through the ripple-carry chain to `p[7]`. The physical design flow produced a DRC- and LVS-clean layout on a $150\,\mu\text{m} \times 150\,\mu\text{m}$ die at a 6.0 ns clock (post-route $F_{\max} = 167.9\,\text{MHz}$), after five runs. The difference from Part C is explained mainly by signoff at the slow corner, and the extra cells (decap, fill, tap, buffers) are what a manufacturable layout needs beyond the logic itself. The results match the expected behaviour of an array multiplier: area that grows as N^2 and delay that grows linearly with N.

8 Contributions

Parts A and B of the assignment were mostly done by Sindhuja. Parts C and D were mostly done by Joanna. Both contributed in writing the report.